第 4 章 数据分析

章节导航

- 了解数据分析常用的库 Pandas，能够读取关系型数据，使用统计、组合、排序、拼接等方法处理数据。
- 了解机器学习与时间序列预测库 scikit-learn。
- 了解图像处理库 OpenCV。

本章知识导图

4.1 数据分析基础

4.1.1 读取与创建数据

进行数据建模前，首先要读取数据。常见的数据存储格式有 JSON(JavaScript object notation，JS 对象简谱)、CSV(comma separated values，逗号分隔值)、XLSX 等。CSV 数据存储格式中每行保存的内容用空格作为分隔符，行与行之间用换行符隔开，CSV 数据可以用常见的表格处理软件打开，视图内容与 XLSX、XLS 格式的文件一样，但是与实际的数据结构有很大差别。使用 CSV 数据存储格式是因为这种类型的数据制作与保存较为方便，仅使用文本编辑器就可以快速编辑。下面是使用 CSV 格式存储电影名称和评分数据的示例。

```
西线无战事 8.6
名侦探柯南：万圣节的新娘 7.9
危笑 6.3
```

类似地，还有 TSV 数据格式，这种类型的数据是用制表符分隔数据，常见于问答系统的标注文件。本质上讲，各种类型的数据交换格式只是为了更加方便地处理数据，使用最基本的 TXT 文件也可以存储与读取文件。

1. 读取数据

在 Python 语言中，Pandas 库用于处理关系型数据。使用 Pandas 库一般都是从导入库开始的，代码如下：

读取文件

```
import pandas as pd          # 为了简便，通常将 pandas 缩写为 pd
```

在如表 4-1 所示的问答对中可以看到，第一列数据是问题，第二列数据是回答，类似这样按行和列划分的数据，可以简单地将其理解为关系型数据。人们通常把关系型数据存储到数据库中。在数据科学建模时，人们通常先把关系型数据导出为 CSV 或 XLSX 文件。

表 4-1　关系型数据：问答对

编号	问题	答案
1	三国演义的作者是谁？	罗贯中
2	兔子的英文是什么？	Rabbit
3	点灯是谁发明的	爱迪生

读取 CSV 文件可以使用 read_csv( )函数，读取 XLSX 文件可以使用 read_excel( )函数，代码如下：

```
import pandas as pd                          # 导入 pandas 库
book=pd.read_csv("book.csv")                 # 使用 read_csv( )函数读取 CSV 文件
student=pd.read_excel("student.xlsx")
```

read_csv( )函数包含很多参数。例如，可以指定 index_col 参数，将 CSV 文件中的某一列作为索引，设置 index_col＝"某一字段"，这样读取数据时不会再创建新的索引；又如，date_parser 参数可以将读取的数据转化为时间格式，这个方法在计算与时间相关的数据时特别重要。刚开始使用 Pandas 库读取数据时，只需指定数据文件所在目录，其余参数使用默认值即可。

读取数据后，可以使用 shape 属性查看读取的数据表的维度。

```
book.shape          # 注意这是一个属性，不带括号
```

还可以使用 head( )方法查看数据表的前 5 行，或使用 tail( )方法查看数据表的后 5 行。

```
book.head( )   # head( )方法可接受一个参数 n，n 表示显示的行数，默认为 5
book.tail( )   # head( )和 tail( )都是方法，需要带括号
```

2. 创建数据

如果要创建新的数据表，可以使用以下代码实现：

```
import pandas as pd
qa=pd.DataFrame({'question':['今天的天气', '兔子的英文怎么说'], 'answer':['晴',
'rabbit']})
```

这里使用了 Pandas 库的十分重要的一种数据类型：DataFrame。DataFrame 是一个数据表，DataFrame 中的每一个条目都是一列，条目的名字就是列名。上面的代码中，创建了一个 DataFrame 类型的数据表，表中包含两列数据，第一列是问题，第二列是答案。

另一种常用的数据类型是 Series，表示数据序列。DataFrame 是数据表，Series 是数据序列，注意不要混淆两种数据类型，这两种数据类型包含的方法是不同的。可以直接传入一个列表来构造一个 Series 对象，代码如下：

```
pd.Series([1, 2, 3, 4, 5])
```

Series 数据序列本质上是 DataFrame 数据表中的一列。

4.1.2　索引与选择

1. 获取单个数据列

在处理数据时，往往需要选取某一列、某一行的数据，或按照某种要求进行数据筛选，Pandas 库为此提供了十分便捷的选择功能。

Pandas 索引数据

4.1.2 节～4.1.6 节使用的数据是某网站上用户对一个影片的评论信息，包含用户的昵称、所在地、粉丝数量，以及评论的发布时间、内容、来源等。首先读取数据文件并输出前 5 行数据，代码如下：

```
# 读取数据
squid=pd.read_csv("Squid_Game_Netflix_Twitter_Data.csv")
squid.head()
```

squid 数据表的前 5 行数据如图 4-1 所示，数据表中包含 user_name、user_location、user_description 等字段。

```
squid.head()
```

	user_name	user_location	user_description	user_created	user_followers	user_friends	user_favourites	user_verified	date
0	the _ûndêr-ratêd niggáh🏀	NaN	@ManUtd die hard 🖤🖤💪💪 \n\n\nYOLO\n\n\nJ'ai b...	2019-09-06 19:24:57+00:00	581	1035	8922	False	2021-10-06 12:05:38+00:00
1	Best uncle on planet earth	NaN	NaN	2013-05-08 19:35:26+00:00	741	730	8432	False	2021-10-06 12:05:22+00:00
2	marcie	NaN	animal crossing. chicken nuggets. baby yoda. s...	2009-02-21 10:31:30+00:00	562	1197	62732	False	2021-10-06 12:05:22+00:00
3	YoMo.Mdp	Any pronouns	Where the heck is the karma\nI'm going on my s...	2021-02-14 13:21:22+00:00	3	277	1341	False	2021-10-06 12:05:04+00:00
4	Laura Reactions	France	I talk and I make reactions videos about shows...	2018-12-19 20:38:28+00:00	330	152	2278	False	2021-10-06 12:05:00+00:00

图 4-1　squid 数据表的前 5 行数据

如果需要获取数据表中某列的数据，可以直接将字段名作为数据表对象的属性来获取

数据。例如，squid 数据表包含 user_name 字段，通过访问 squid. user_name 即可获取该列数据。输出结果如下：

```
0        the_ûnder-rat è d nigg áh
1        Best uncle on planet earth
2        marcie
3        YoMo. Mdp
4          Laura Reactions
......
29995      Lu Xu(Lou Shoe/徐露)
29996      See Ya, Sucker
29997      maddy earley
29998
29999      White Thunder
Name:user_name, Length:30000, dtype:object
```

另一种数据索引的方式是以字典的方式，Python 语言有字典类型的数据，DataFrame 类型的数据可以用键值对的方式索引，如 squid["user_name"]，该方法与第一种索引方法的结果相同。

上述两种获取列数据的方法，从语法上来看没有任何差别，在实际的数据科学编程中也经常看到两种方法的使用，但是以字典的方式进行索引的方法可以获取一些名称比较复杂的字段，如名称中带有空格的字段。例如，user name 字段，user 与 name 之间用空格分隔，如果使用第一种方法，squid. user name 是无法获取该字段的，只能使用第二种方法 squid["user name"]。

使用 squid["user_name"]获取的是所有的用户名，如果仅需获取第一个用户名，则可以在其后增加一个索引序号，代码如下：

```
squid["user_name"][0]
```

输出结果如下：

```
'the _ûnd ër- rat è d nigg á h'
```

Pandas 库中数据索引的方式与 Python 语言中索引列表或字符串的方式相同。

2. 获取多个数据列

在数据处理的过程中，在原始表格过大的情况下，需要裁剪出数据表中的某些列，可以使用[ ]以字典的方式获取数据。例如，要同时获取 squid 数据表中的 user_name 和 user_location 字段，代码如下：

```
squid[["user_name","user_location"]]  # 注意此处使用了两个方括号
```

输出结果如下：

```
        user_name                          user_location
0       the_under-ratedniggah              NaN
1       Best uncle on planet earth         NaN
2       marcie                             NaN
3       YoMo. Mdp                  Any pronouns
4       Laura Reactions            France
......
29995    Lu Xu(Lou Shoe/徐露)       Halifax, Nova Scotia
29996    See Ya, Sucker             724☠
29997    maddy earley               NaN
29998
29999    White Thunder              Somewhere
30000 rows×2 columns
```

需要注意的是，上述语句使 squid 数据表接收一个列表作为输入，返回多列数据组成的 DataFrame 数据表，因此数据的获取形式变为[[ ]]，看上去有些特殊，如果仅使用[]会报错，提示 KeyError：('user _ name'，'user _ location')，即没有这个键，因为程序将("user _ name","user _ location")看成一个键。

3. 基于索引选择数据

基于索引选择数据使用 iloc 属性，格式为 iloc[n]，想选择数据的第几行，就将 n 设置为该行的索引序号。例如，选择 user_description 列的第 4 行数据，代码如下：

```
squid["user_description"].iloc[3]
```

输出结果如下：

```
"Where the heck is the karma\ nI'm going on my school grave brb\ n# Technosupport"
```

iloc 属性默认是按行进行数据筛选，如果要选择某一列的数据，会相对麻烦一些，代码如下：

```
squid.iloc[:, 3]  # 选择第 4 列数据
```

输出结果如下：

```
0          2019-09-06 19:24:57+00:00
1          2013-05-08 19:35:26+00:00
2          2009-02-21 10:31:30+00:00
3          2021-02-14 13:21:22+00:00
4          2018-12-19 20:38:28+00:00
......
```

```
29995     2015-07-24 13:42:51+00:00
29996     2014-02-06 04:14:20+00:00
29997     2014-11-27 04:32:52+00:00
29998     2021-03-04 11:54:39+00:00
29999     2013-09-05 18:32:49+00:00
Name:user_created, Length:30000, dtype:object
```

使用上述方法的结果与使用 squid["user_created"]方法的结果相同。在上述方法中，注意到使用了冒号(:)，该符号也属于 Python 语法，表示所有内容，当冒号与其他数值一起使用时，表示选择一定范围内的数据。例如，选择 user_name 列的第 1、2、3 行数据，代码如下：

```
squid.iloc[:3, 0]
```

输出结果如下：

```
0    the_ûndër- rat è d nigg á h
1    Best uncle on planet earth
2    marcie
Name:user_name, dtype:object
```

在 Python 语言中，负数代表从结尾开始索引，这在 Pandas 库中同样适用。选择整个数据表的最后 5 行数据的代码如下：

```
squid.iloc[- 5:]
```

输出结果如图 4-2 所示。

	user_name	user_location	user_description	user_created	user_followers	user_friends	user_favourites	user_verified	date
29995	Lu Xu(Lou Shoe/徐露)	Halifax, Nova Scotia	Journalist @the_logic | Podcaster @CKDU881FM |...	2015-07-24 13:42:51+00:00	309	869	2887	False	2021-10-10 01:43:40+00:00
29996	See Ya, Sucker	724	NaN	2014-02-06 04:14:20+00:00	105	136	971	False	2021-10-10 01:43:25+00:00
29997	maddy earley	NaN	stay alive, friend. It's worth it.	2014-11-27 04:32:52+00:00	44	119	3135	False	2021-10-10 01:42:40+00:00
29998	S	*"Ahsen Eroglu"*	Alperen Duymaz owns me ♡ **DICBAR** | İlhan ♡\n ...	2021-03-04 11:54:39+00:00	479	153	44597	False	2021-10-10 01:42:32+00:00
29999	White Thunder	Somewhere	Royal captain of the Faire Army, Novelist, art...	2013-09-05 18:32:49+00:00	21	44	1435	False	2021-10-10 01:42:30+00:00

图 4-2　使用 iloc( )方法选择整个数据表的最后 5 行数据的输出结果

4. 基于标签选择数据

基于标签选择数据使用 loc 属性，如选择 user_name 列的第 2 行数据，代码如下：

```
squid.loc[1, 'user_name']
```

输出结果如下：

```
'Best uncle on planet earth'
```

squid.loc[1，'user _ name']的第一个参数为 1，这里的 1 表示数据表的行标签，如果 1 出现在 iloc 属性中，则表示索引序号。

loc 属性常常与 iloc 属性混淆。在使用 iloc 属性时，忽略了数据表的标签，也就是字段名和行名，直接将数据表当成一个矩阵，通过元素所在的行和列进行选择。通常情况下，在选择列数据时，每列都带有字段名，因此使用 loc 属性。

5. 条件筛选

在 Pandas 库中，可以使用条件语句对数据进行筛选，类似于 SQL 中 SELECT A FROM B WHERE CONDITION 语句的功能，但使用 Pandas 库的条件语句要方便得多。例如，需要筛选用户位置在法国的用户信息，首先需要设置筛选条件，判断用户是否在法国，代码如下：

```
squid["user_location"] =="France"
```

输出结果如下：

```
0          False
1          False
2          False
3          False
4          True
......
29995      False
29996      False
29997      False
29998      False
29999      False
Name:user_location, Length:30000, dtype:bool
```

可以看到，输出结果是一系列布尔值，当布尔值为 True 时，表示当前行满足筛选条件，当布尔值为 False 时，表示当前行不满足筛选条件。然后根据输出结果，使用 loc( )方法筛选出布尔值为 True 的数据，代码如下：

```
squid.loc[squid["user_location"]=="France"]
```

使用 loc( )方法筛选的输出结果如图 4-3 所示。

	user_name	user_location	user_description	user_created	user_followers	user_friends	user_favourites	user_verified
4	Laura Reactions	France	I talk and I make reactions videos about shows...	2018-12-19 20:38:28+00:00	330	152	2278	False
1539	Laura Reactions	France	I talk and I make reactions videos about shows...	2018-12-19 20:38:28+00:00	330	152	2278	False
6022	Nath L.	France	Le monde appartient à ceux qui rêvent trop!#It...	2010-07-25 20:14:08+00:00	1351	1146	24404	False
7074	Alcmène	France	#CastMember à @DisneylandParis depuis 2006: F...	2009-06-17 08:45:58+00:00	703	199	8638	False
7568	Sach'imi	France	Graphiste Freelance in #ILLUSTRATION #MOTION\n...	2019-07-02 12:49:16+00:00	16	152	197	False

图 4-3　使用 loc( )方法筛选的输出结果

包含多个筛选条件的筛选方法也经常用到，多个筛选条件之间使用“与”“或”“非”关联。例如，想要筛选出用户位置在法国，且粉丝数量大于 100 的用户，代码如下：

```
squid.loc[(squid["user_location"]=="France")&(squid["user_followers"]>100)]
```

使用多个条件筛选的输出结果如图 4-4 所示。

	user_name	user_location	user_description	user_created	user_followers	user_friends	user_favourites	user_verified
4	Laura Reactions	France	I talk and I make reactions videos about shows...	2018-12-19 20:38:28+00:00	330	152	2278	False
1539	Laura Reactions	France	I talk and I make reactions videos about shows...	2018-12-19 20:38:28+00:00	330	152	2278	False
6022	Nath L.	France	Le monde appartient à ceux qui rêvent trop!#It...	2010-07-25 20:14:08+00:00	1351	1146	24404	False
7074	Alcmène	France	#CastMember à @DisneylandParis depuis 2006: F...	2009-06-17 08:45:58+00:00	703	199	8638	False
10081	SUSSUCRE	France	Créateur de contenu | #TeamOL | narutoblazing	2013-07-10 10:22:12+00:00	27607	414	45626	False

图 4-4　使用多个条件筛选的输出结果

在代码中，& 表示“与”，｜表示“或”，～表示“非”。

Pandas 库中还内置了很多选择器，如果筛选条件是某个字段的值为一个列表中的任一元素，可以使用 isin( )方法。例如，筛选用户位置在德国、法国或美国的用户，代码如下：

```
squid[squid["user_location"].isin(['France', 'Germany', 'United States'])]
```

使用 isin( )方法筛选的输出结果如图 4-5 所示。

	user_name	user_location	user_description	user_created	user_followers	user_friends	user_favourites	user_verified
4	Laura Reactions	France	I talk and I make reactions videos about shows...	2018-12-19 20:38:28+00:00	330	152	2278	False
26	EarlyGame	Germany	Own Your Game	2019-11-13 14:56:55+00:00	49381	153	963	False
47	MyMasqueBar - Self-Care is the Best Care	United States	We are everything masks. Tag #MyMasqueBar to b...	2013-09-26 15:47:33+00:00	561	700	900	False
111	Businessbea	United States	📇 | Educational business content. ☑ | Help oth...	2021-06-01 12:41:49+00:00	138	317	54	False
382	Cuban Spice (The 6th Spice Girl) 😉cu	United States	I do be the preediest preediest girl, I do be ...	2009-04-24 14:11:19+00:00	317	1969	20403	False

图 4-5　使用 isin( )方法筛选的输出结果

4.1.3　统计与映射

1. 基本统计方法

常用统计方法

Pandas 库中有很多统计方法供用户使用，使用这些方法能够快速获得数据的统计信息，如平均值、方差、标准差等统计量。例如，想要获取用户粉丝数的平均值、最大值、最小值等，可以使用 describe( )方法，代码如下：

```
squid["user_followers"].describe( )
```

输出结果如下：

```
count      3.000000e+04
mean       1.565457e+04
std        2.525806e+05
min        0.000000e+00
25%        7.400000e+01
50%        3.190000e+02
75%        1.171000e+03
max        1.684642e+07
Name:user_followers, dtype:float64
```

可以看到，describe( )方法给出了用户粉丝数的平均值、标准差、最小值、上四分位数、下四分位数等统计量，通过该方法可以对数据的分布有初步认识。

如果要单独查看某个统计量，可以直接调用计算该统计量的方法，如计算平均值的代码如下：

```
squid["user_followers"].mean( )
```

另一个较为有用的统计方法是 info( )，该方法能够输出每列数据的非空值的数量、数据类型，以及整个数据表的内存占用信息等。在数据建模时，一般要求输入特征不能为空，可以使用 info( )方法查看，代码如下：

```
squid.info( )
```

输出结果如下：

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex:30000 entries, 0 to 29999
Data columns(total 12 columns):
  #   Column            Non- Null Count  Dtype
---  ------            --------------   -----
  0  user_name         29999 non- null  object
  1  user_location     21378 non- null  object
  2  user_description  28186 non- null  object
  3  user_created      30000 non- null  object
  4  user_followers    30000 non- null  int64
  5  user_friends      30000 non- null  int64
  6  user_favourites   30000 non- null  int64
  7  user_verified     30000 non- null  bool
  8  date              30000 non- null  object
  9  text              30000 non- null  object
  10  source           30000 non- null  object
  11  is_retweet       30000 non- null  bool
dtypes:bool(2), int64(3), object(7)
memory usage:2.3+MB
```

注意：仅能对 DataFrame 对象使用 info( )方法，不能对 Series 对象使用该方法。

unique( )方法用于查看某一列数据中有多少个不同的数据。例如，查看用户发布评论所使用的设备有哪些，代码如下：

```
squid["source"].unique()
```

输出结果如下：

```
array(['Twitter for Android', 'Twitter Web App', 'Twitter for iPhone',
       'Buffer', 'Twitter for iPad', 'PromoRepublic', 'TweetDeck',
       'Instagram', 'Sprout Social', 'Hootsuite Inc.', 'Tweetbot for iOS',
       'dlvr.it', 'Meltwater Social', 'Twitter for Mac', ..., dtype=object)
```

value_counts( )方法的作用是统计某个特征字段的不同取值的个数，该方法常用于性别统计、星座统计等。例如，对用户发布评论所使用的设备进行统计，代码如下：

```
squid["source"].value_counts()
```

输出结果如下：

```
Twitter for iPhone          13919
Twitter for Android          8851
Twitter Web App              5251
Twitter for iPad              690
TweetDeck                     354
......
TheWrap                         1
Xbox One Social                 1
Oktopost                        1
TwitPane for Android            1
BizToc                          1
Name:source, Length:133, dtype:int64
```

2. 映射

在进行数据分析时，往往会对数据进行一定程度的修改，以方便后续的建模和分析，例如，将性别为男的数据映射为 0，将性别为女的数据映射为 1。又如，对学生的成绩进行打分，将 90 分以上的数据映射为 A，将 80～90 分的数据映射为 B 等。上述操作意味着要对数据表中的数据进行批量化处理，使用 Python 语言中的 for 循环可以实现上述操作，但太过烦琐。批量化处理意味着对所有数据进行相同的操作，在 Pandas 库中有专门的方法，这就是本节介绍的映射方法。

apply( )方法是 Pandas 库中功能十分强大的映射方法，该方法能够按照某一条件对某列数据进行映射。例如，根据 user_verified 字段，创建新的字段 verified，若 user_verified 字段为 False，则设置为 0，否则设置为 1，代码如下：

```
def check_verify(entry):
    if entry["user_verified"] is False:
        return 0
    else:
        return 1
squid["verified"]=squid.apply(check_verify, axis=1)
squid["verified"].value_counts( )
```

输出结果如下：

```
0    28987
1    1013
Name:verified, dtype:int64
```

分析上述代码，首先定义了 check_verify( )函数，该函数接收一条数据作为输入，当该条数据中 user_verified 字段为 False 时，返回 0，否则返回 1。然后，squid.apply( )方法调用了这个函数，并将结果赋值给新的列 squid["verified"]。调用时 check_verify 不能带括号，如果带上括号就是将函数的返回结果作为输入，发送给 apply( )方法。

此外，squid.apply(check_verify，axis＝1)中参数 axis 的取值是 0 或者 1，0 代表按照每列进行计算，1 代表按照每行进行计算，此处是每次取一行数据，然后判断指定字段是 True 还是 False，因此参数 axis 需要设置为 1。

4.1.4 组合与排序

1. 组合

4.1.3 节中，使用 value_counts( )方法获得了对单列数据的频次统计，还可以使用 groupby( )方法实现上述功能，代码如下：

```
squid.groupby("source").source.count( )
```

输出结果如下：

```
15 Minute Fun          2
AT Auto Tweet          1
Agorapulse app         6
Apphi                  1
Bearguin               2
......
sankoffoncor           1
tshell- bash           1
tweetpancake           1
twitterproject122      1
www.keengamer.com      1
Name:source, Length:133, dtype:int64
```

groupby( )方法的功能与 SQL 中的聚合操作一致，都是对某一字段进行聚合统计，在上述代码中，根据 source 对用户进行分组，然后获取每组用户的 source 字段的数据，再使用 count( )方法统计其出现的次数。

也可以使用其他的聚合方法，如统计每种设备上粉丝数最多的用户的粉丝数，可以使用 max( )方法，代码如下：

```
squid.groupby("source").user_followers.max()
```

输出结果如下：

```
15 Minute Fun            269
AT Auto Tweet            361
Agorapulse app      1585950
Apphi                  8645
Bearguin               8070
......
sankoffoncor              7
tshell- bash           3814
tweetpancake             63
twitterproject122        16
www.keengamer.com     10408
Name:user_followers, Length:133, dtype:int64
```

2. 排序

对数据进行聚合统计后，得到的统计数据没有按照大小顺序排列，使用 sort_values( ) 方法可以对数据排序，代码如下：

```
max_followers=squid.groupby("source").user_followers.max()
max_followers.sort_values()
```

输出结果如下：

```
source
recurpost.com                              1
EveryoneSocial                             1
Microsoft Azure Logic Apps                 1
sankoffoncor                               7
Outfy Inc                                 14
......
TweetDeck                            8237104
Sprinklr                             9066758
Twitter Web App                      9207638
Buffer                              16845024
Twitter Media Studio                16846417
Name:user_followers, Length:133, dtype:int64
```

sort_values( )方法默认将数据从小到大排序，若要从大到小排序，可以设置参数 ascending＝False。此外，如果按照多个字段排序，可以将这些字段名称组成列表，使用 by

参数传入列表，代码如下：

```
squid.sort_values(by=["user_followers","user_favourites"])
```

sort_values( )方法的输出结果如图 4-6 所示。

	user_name	user_location	user_description	user_created	user_followers	user_friends	user_favourites	user_verified
282	George .	Athens, NY	Sarcasm is my home. 👀	2021-10-05 17:04:50+00:00	0	1	0	False
349	ToDisenyo	NaN	Design, Marketing, Innovation	2021-02-19 10:23:21+00:00	0	14	0	False
616	IBADO	Metro Manila, Philippines	Selling locally manufactured brand new clothes...	2021-10-06 08:34:24+00:00	0	0	0	False
867	PixelQuack NFT	OpenSea	NFT Creator, check out my collections availabl...	2021-10-06 08:55:46+00:00	0	0	0	False
1150	IBADO	Metro Manila, Philippines	Selling locally manufactured brand new clothes...	2021-10-06 08:34:24+00:00	0	0	0	False

图 4-6　sort _values( )方法的输出结果

4.1.5　数据类型与缺失值

Pandas 库中的数据有多种类型，使用 dtypes 属性可以获取 DataFrame 数据表中每一列的数据类型，代码如下：

```
squid.dtypes
```

输出结果如下：

```
user_name                object
user_location            object
user_description         object
user_created             object
user_followers           int64
user_friends             int64
user_favourites          int64
user_verified            bool
date                     object
text                     object
source                   object
```

```
is_retweet                bool
verified                  int64
dtype:object
```

此外，如果查看 Series 对象的数据类型，则需要使用 Series.dtype 属性。

使用 astype( )方法，可以更改数据类型，在处理包含相同字段的多个数据表时，常常会遇到不同表中同一字段名的数据类型不一致的问题。可以使用 astype( )方法修改数据的类型。这种修改方法会对一个数据表中的某一列进行批量转换，而非对单个数值进行转换。例如，将 user_followers 字段的数据类型改为 int32，代码如下：

```
squid["user_followers_32"]=squid["user_followers"].astype("int32")
squid["user_followers_32"].dtype
```

输出结果如下：

```
dtype('int32')
```

在 Pandas 库中，可以很方便地查看与处理空值，isnull( )方法用于查看数据是否存在空值，fillna( )方法常用于填充缺失值。例如，查看用户所在位置为空的数据，并将空值填充为 Unknown，代码如下：

```
unknown_loc =squid[squid["user_location"].isnull( )]["user_location"]
unknown_loc.fillna('Unknown')
```

输出结果如下：

```
0          Unknown
1          Unknown
2          Unknown
6          Unknown
8          Unknown
......
29981      Unknown
29986      Unknown
29987      Unknown
29990      Unknown
29997      Unknown
Name:user_location, Length:8622, dtype:object
```

4.1.6　数据拼接

在处理数据时，数据可能来自多个数据源。为了将多个数据源的数据拼接起来，可以

使用 pd.concat( )函数，代码如下：

```
df1=squid.iloc[0:2,:]
df2=squid.iloc[5:7,:]
pd.concat([df1, df2])
```

数据拼接的输出结果如图 4-7 所示。

	user_name	user_location	user_description	user_created	user_followers	user_friends	user_favourites	user_verified	date
0	the _ûndër-ratëd niggáh	NaN	@ManUtd die hard \n\n\nYOLO\n\n\nJ'ai b...	2019-09-06 19:24:57+00:00	581	1035	8922	False	2021-10-06 12:05:38+00:00
1	Best uncle on planet earth	NaN	NaN	2013-05-08 19:35:26+00:00	741	730	8432	False	2021-10-06 12:05:22+00:00
5	Peyman	United Kingdom	Official @KardiaChain $KAI Ambassador\nMarketi...	2018-01-27 12:07:31+00:00	546	318	6265	False	2021-10-06 12:04:54+00:00
6	Aeriaaaa♡	NaN	Fujoshi / Thai BL-obsessed/Always distracted ...	2021-06-01 14:08:10+00:00	14	110	518	False	2021-10-06 12:04:45+00:00

图 4-7　数据拼接的输出结果

第一列的标签是 0、1、5、6，分别对应源数据的第 1 行、第 2 行、第 6 行、第 7 行数据。也就是说，组合之后的结果会保留原来的编号，不会重新编号。如果设置参数 ignore_index 为 True，则拼接后的结果会重新编号，代码如下：

```
df1=squid.iloc[0:2,:]
df2=squid.iloc[5:7,:]
pd.concat([df1, df2], ignore_index=True)
```

输出结果如图 4-8 所示。

	user_name	user_location	user_description	user_created	user_followers	user_friends	user_favourites	user_verified	
0	the _ûndër-ratëd niggáh	NaN	@ManUtd die hard \n\n\nYOLO\n\n\nJ'ai b...	2019-09-06 19:24:57+00:00	581	1035	8922	False	1
1	Best uncle on planet earth	NaN	NaN	2013-05-08 19:35:26+00:00	741	730	8432	False	1
2	Peyman	United Kingdom	Official @KardiaChain $KAI Ambassador\nMarketi...	2018-01-27 12:07:31+00:00	546	318	6265	False	1
3	Aeriaaaa♡	NaN	Fujoshi / Thai BL-obsessed/Always distracted ...	2021-06-01 14:08:10+00:00	14	110	518	False	1

图 4-8　数据拼接后重新编号的输出结果

数据分析基础总结：本节主要介绍了数据科学工具库 Pandas 的基本使用，Pandas 库主要用于操作关系型数据，使用 Pandas 库可以很方便地从持久化存储格式中读取数据，如数据库、CSV 文件、TXT 文件等。Pandas 库内置了很多计算工具，方便用户进行数据分析。在数据工程中，往往会先使用 Pandas 库读取数据，对数据做预处理，如对数据进行统计分析，再对部分数据进行映射、聚合、填充缺失值等操作，方便后续将数据输入模型进行训练。

除了阅读本书学习 Pandas 库外，推荐有英文阅读基础的读者查找 Pandas 库的官方文档，阅读更详细的函数说明。例如，pd.concat( )函数的默认参数还包括 axis、join、levels 等，每个参数在官方文档中都有十分详细的说明，阅读官方文档能够快速提升使用 Pandas 库的熟练程度。

4.2 机器学习库 scikit-learn

本节介绍 Python 语言的机器学习库 scikit-learn。scikit-learn 库是开源数据科学建模工具，主要用于构建分类模型、回归模型和聚类模型。当数据特征较多，模型无法拟合数据时，可以使用 scikit-learn 库做数据降维，以降低数据冗余度，scikit-learn 库还可用于数据预处理和机器学习模型融合。

4.2.1　构建分类模型

分类任务是数据科学分析建模中的一种有监督学习的任务，常见的分类任务如基于图像的猫狗分类、垃圾邮件分类、人脸识别等。基于 scikit-learn 库可以对关系型数据构建分类模型，一般流程如下：

(1)使用 Pandas 库等工具读取关系型数据，进行数据探索、缺失值填充；

(2)使用 scikit-learn 库对数据特征进行归一化；

(3)使用 scikit-learn 库将读取的数据拆分为训练集和测试集，其中训练集用于模型拟合数据，测试集用于验证模型的泛化性。

下面以鸢尾花数据集为例，使用支持向量机(support vector machine，SVM)算法，构建一个分类模型。鸢尾花数据集是一份经典的机器学习数据集，常被用于分类问题的研究和算法性能评估。鸢尾花数据集包含 3 个不同种类的鸢尾花：山鸢尾(iris-setosa)、变色鸢尾(iris-versicolor)和维吉尼亚鸢尾(iris-virginica)。每个样本包含 4 个特征：花萼长度(sepal length)、花萼宽度(sepal width)、花瓣长度(petal length)和花瓣宽度(petal width)。该数据集常用于初学者了解机器学习算法的基本概念和流程，也常被用来作为算法性能比较和验证的基准数据集。下面的代码使用 Matplotlib 库和 scikit-learn 库可视化鸢尾花数据集，输出结果如图 4-9 所示。

```
import matplotlib.pyplot as plt
from sklearn import datasets
# 加载鸢尾花数据集
iris=datasets.load_iris( )
```

```
X=iris.data
y=iris.target
# 可视化数据集中的前两个特征
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis')    plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.title('Iris Dataset')
plt.show()
```

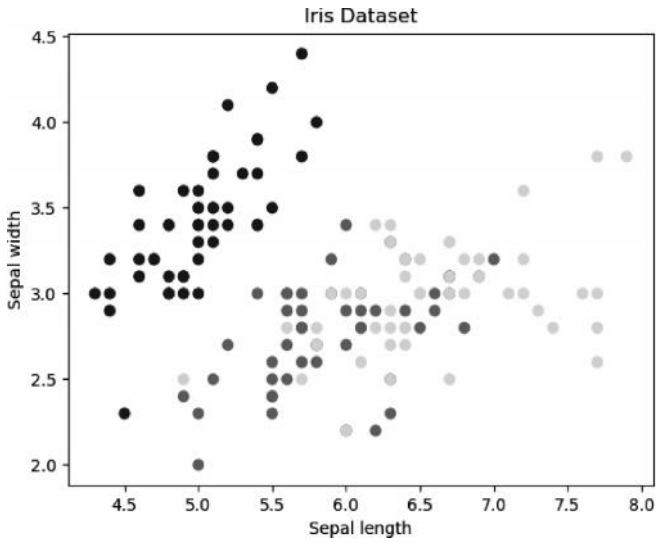

图 4-9　鸢尾花数据集中花萼长度和花萼宽度的分布

首先导入 scikit-learn 库和 NumPy 库，代码如下：

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

然后加载鸢尾花数据集，使用 train_test_split( )函数将数据拆分为训练集和测试集，创建一个 StandardScaler 类型的对象，分别使用其 fit_transform( )方法和 transform( )方法将训练集和测试集的数据标准化，以确保每个特征的均值为 0、标准差为 1，代码如下：

```
# 加载鸢尾花数据集
iris=datasets.load_iris()
X=iris.data
y=iris.target
```

```
# 将数据集分成训练集和测试集
X_train, X_test, y_train, y_test=train_test_split(X, y, test_size=0.2, random_
state=42)
# 标准化数据
scaler=StandardScaler( )
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
```

创建一个 SVM 分类器，即 SVC 类型的对象，并使用训练集数据进行训练。最后使用测试集数据进行预测，并计算分类器的准确率(accuracy)，代码如下：

```
svm=SVC(kernel='linear', C=1, random_state=42)
# 使用训练集数据进行训练
svm.fit(X_train, y_train)
# 对测试集数据进行预测
y_pred=svm.predict(X_test)
# 计算分类器的准确率
accuracy=accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

4.2.2　构建回归模型

机器学习中的回归任务是指预测连续值的预测性建模问题，其目标是为输入变量和连续输出变量之间建立一个函数关系。回归任务通常用于预测数值型数据，如房价、股票价格、销售额等。

回归任务与分类任务都是机器学习中的常见任务，但它们的目标不同。分类任务的目标是根据输入变量确定样本属于哪个类别，而回归任务的目标是预测输入变量和输出变量之间的函数关系。

以下是一个使用 scikit-learn 库进行回归任务的简单示例。这个示例中将使用加州房价数据集，该数据集是一个经典的数据集，通常用于回归问题的分析。

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn. metrics import mean_squared_error
from sklearn.datasets import fetch_california_housing

# 获取加州房价数据集
housing= fetch_california_housing()
X= housing.data
y= housing.target
```

```
# 将数据集分成训练集和测试集
X_train, X_test, y_train, y_test= train_test_split(X, y, test_size= 0.2,
random_state= 42)

# 创建线性回归模型
regressor= LinearRegression()
# 使用训练数据对模型进行训练
regressor.fit(X_train, y_train)
# 对测试数据进行预测
y_pred= regressor.predict(X_test)

# 计算模型的均方误差
mse= mean_squared_error(y_test, y_pred)
print("Mean squared error:", mse)
```

4.2.3 构建聚类模型

构建聚类模型使用的是聚类算法。聚类算法是机器学习中的一种无监督学习方法，用于将数据集中的数据分成不同的聚类或组。聚类算法的目标是将相似的数据点分组在一起，使得同一聚类中的数据点相似度高，而不同聚类中的数据点相似度低。

以 k 均值(k-means)聚类算法为例，下面的代码首先使用 scikit-learn 库中的 make_blobs( ) 函数创建了一个包含 300 个样本和 4 个中心的模拟数据集。其次，创建一个 KMeans 聚类器，即 KMeans 类型的对象，并设置聚类数量为 4。然后，使用拟合函数对数据进行聚类拟合，并获取每个数据点的聚类标签。最后，使用 Matplotlib 库将聚类结果可视化，如图 4-10 所示。读者可以根据自己的实际需求对模型进行调整和优化。

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

# 使用 make_blobs( )函数创建模拟数据集
X, y=make_blobs(n_samples=300, centers=4, random_state=42, cluster_std=0.60)

# 创建 KMeans 模型，设置聚类数量为 4
kmeans=KMeans(n_clusters=4, random_state=42)

# 使用拟合函数对数据进行聚类拟合
kmeans.fit(X)

# 获取每个数据点的聚类标签
labels=kmeans.labels_

# 绘制聚类结果的可视化图形
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.show()
```

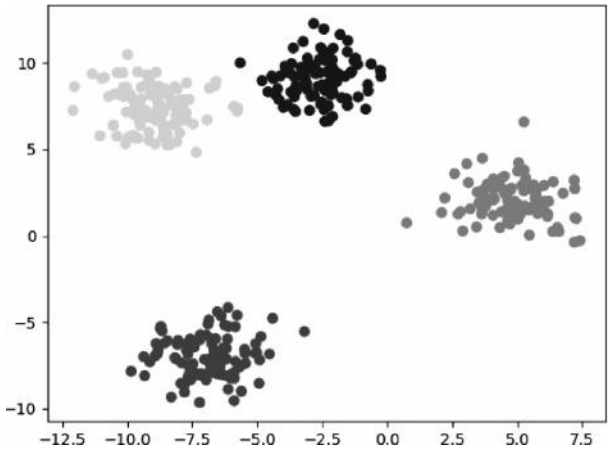

图 4-10　聚类结果可视化

4.3 图像处理库 OpenCV

OpenCV 库是一个开源的计算机视觉库，为开发者提供了一系列用于处理图像和视频的函数与工具。它被广泛应用于计算机视觉、机器学习和人工智能领域。它还可以与其他流行的机器学习库，如 TensorFlow 和 PyTorch 等进行集成。OpenCV 库具有广泛的应用领域，它被用于物体检测和识别、人脸分析和识别、手势识别、运动跟踪。在 Python 语言中，也可以使用 pillow、scikit-image 等库处理图像。

4.3.1　图像的读取与保存

在 Python 程序中使用 OpenCV 库时，需要通过 cv2 导入该库，可使用其中的 imread( ) 函数读取图像，使用 imshow( )函数显示图像，代码如下：

```
import cv2
# 读取图像
img=cv2.imread('lena.jpg')
# 显示图像
cv2.imshow('image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
# 保存图像
cv2.imwrite('new_image.jpg', img)
```

在上述代码中，首先通过名字 cv2 导入 OpenCV 库，然后使用 imread( )函数读取名为

lena.jpg 的图像文件，并将其存储在 img 变量中。接下来，使用 imshow( )函数将图像显示在屏幕上，如图 4-11 所示，并使用 waitKey( )函数等待用户按下任意键，最后使用 destroyAllWindows( )函数关闭所有打开的窗口。为了保存图像，可以使用 imwrite( )函数将 img 变量中的图像保存为新图像文件 new_image.jpg。

图 4-11　使用 OpenCV 库读取图像并显示

4.3.2　图像的基本信息

使用 OpenCV 库读取的图像包含一些基本信息，常见的彩色图像有三个色彩通道，分别是 R(红)、G(绿)、B(蓝)，而灰度图像仅有一个色彩通道。对于灰度图像，每个像素只有一个颜色通道值；对于彩色图像，每个像素有三个颜色通道值(红、绿、蓝)。下面的代码使用 OpenCV 库获取图像的宽度和高度。

```
import cv2
# 读取图像
img=cv2.imread('image.jpg')
# 获取图像大小
height, width=img.shape[:2]
print('图像大小:', height, width)
```

图像的像素值大小取值在 0～255 之间。如图 4-12 所示，图像中帽子处的像素值为[58，66，133]，头发处的像素值为[23，40，83]，图像的像素值越接近 0，表示当前位置越暗，图像的像素值越接近 255，表示当前位置越亮。

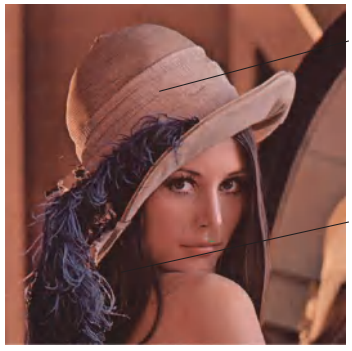

图 4-12　不同位置的像素值大小

4.3.3　图像的平移与旋转变换

图像的变换通常用于数据增强，能够提升模型的泛化性。要平移图像，可以使用 cv2.warpAffine( )函数，代码如下：

```
import cv2
import numpy as np
# 读取图像
img=cv2.imread('lena.jpg')
# 定义平移矩阵，这里将图像向右移动 100 像素，向下移动 50 像素
M=np.float32([[1, 0, 100], [0, 1, 50]])
# 对图像进行平移变换
dst=cv2.warpAffine(img, M, (img.shape[1]+100, img.shape[0]+50))
# 显示平移后的图像
cv2.imshow('image', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

要旋转图像，可以使用 cv2.getRotationMatrix2D()函数定义旋转矩阵，然后将其与原始图像进行矩阵乘法运算，代码如下：

```
import cv2
# 读取图像
img=cv2.imread('lena.jpg')
# 定义旋转矩阵，这里将图像顺时针旋转 45 度
M=cv2.getRotationMatrix2D((img.shape[1]/2, img.shape[0]/2), - 45, 1)
# 对图像进行旋转变换
dst=cv2.warpAffine(img, M, (img.shape[1], img.shape[0]))
# 显示旋转后的图像
cv2.imshow('image', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

图像的平移与旋转变换结果，如图 4-13 所示。

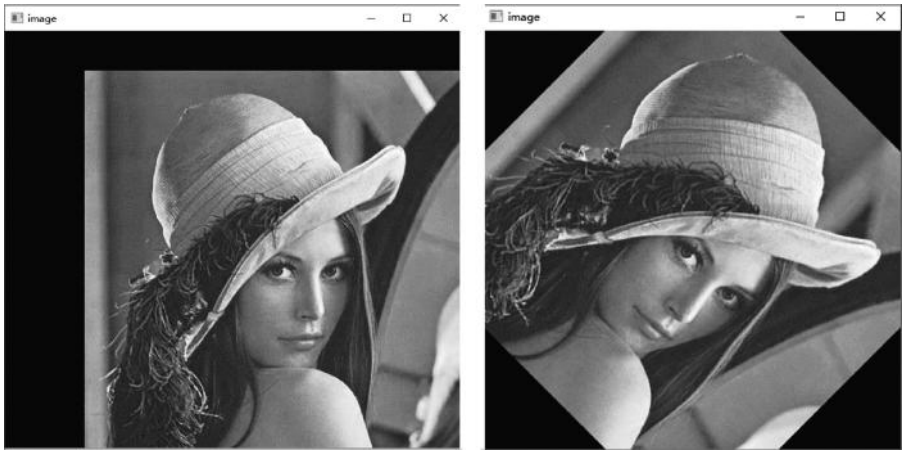

图 4-13　图像的平移与旋转变换结果

4.3.4 图像的边缘信息提取

要使用 OpenCV 库提取图像边缘信息，可以使用 Canny 边缘检测算法。Canny 边缘检测算法是一种经典的边缘检测算法，它可以检测出图像中的强边缘，并抑制弱边缘，从而得到清晰的边缘图像，代码如下：

```
import cv2

# 以灰度模式读取图像
img=cv2.imread('lena.jpg', cv2.IMREAD_GRAYSCALE)
# 使用 Canny 边缘检测算法提取边缘信息
edges=cv2.Canny(img, 100, 200)
# 显示原始灰度图像和边缘信息图像
cv2.imshow('Original Image', img)
cv2.imshow('Edge Image', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

在上述代码中，首先使用 cv2.imread()函数读取名为 lena.jpg 的图像文件，并将其转换为灰度图像。然后使用 cv2.Canny()函数对图像进行 Canny 边缘检测，其中第二个和第三个参数分别为像素的低阈值和高阈值。低阈值用于边缘连接，高阈值用于强边缘检测。最后使用 cv2.imshow()函数显示原始灰度图像和边缘信息图像，如图 4-14 所示。

图 4-14　原始灰度图像和边缘信息图像

习　　题

1. Python 语言中的图像处理工具除了 OpenCV 库之外，还有 pillow 工具库。使用命令 pip install pillow 安装该库，并使用 Image.open()函数读取图像，与 OpenCV 工具库的

cv2.imread( )函数读取的图像进行对比，两者读取的图像数据格式是否一致？从图像大小、格式、颜色通道的角度分别进行比较。

2. 使用 Python 语言的 Pandas 工具库，可以对关系型数据进行排序，使用 pandas.DataFrame.sort_values( )方法时，参数 inplace 的作用是什么？

3. Pandas 工具库不仅能从 XLS、XLSX 等文件中读取数据，还能连接数据库获取数据。下面是使用 Pandas 库从数据库中读取数据的代码，尝试使用 MySQL 创建一个数据库 StuDB，库中包含一张学生的基础信息表 stu_info，运行下列代码，并从数据库中获取 stu_info 表的全部数据。

```
import pandas as pd
from sqlalchemy import create_engine
# 初始化数据库连接
# 按实际情况依次填写 MySQL 的用户名、密码、IP 地址、端口、数据库名
engine=create_engine('mysql+pymysql://用户名:密码@localhost:3306/StuDB')
# 填写自己所需的 SQL 语句，可以是复杂的查询语句
sql_query='select * from stu_info; '
# 使用 Pandas 库的 read_sql_query()函数执行 SQL 语句，并存入 DataFrame
df_read=pd.read_sql_query(sql_query, engine)
print(df_read)
```

4. 尝试拍摄一张照片，并使用 OpenCV 库的 Canny 边缘检测算法提取拍摄照片的边缘信息。在提取图像边缘信息时，有以下两个参数可以设置。

在 Canny 算法中，minVal(最小阈值)和 maxVal(最大阈值)是两个关键参数，它们用于确定边缘的强度。这两个阈值在算法的滞后阈值处理步骤中起到决定性作用。

(1)minVal(最小阈值)：这个值定义了边缘强度的下限。如果一个像素的梯度值低于这个阈值，它将不会被考虑为边缘。

(2)maxVal(最大阈值)：这个值定义了边缘强度的上限。如果一个像素的梯度值高于这个阈值，它将被确定为强边缘。

调整 minVal 和 maxVal，查看不同阈值计算的包含边缘信息的图像差异。

5. 在 4.2.1 节中，使用 scikit-learn 库的 SVM 分类器构建了分类模型，常见的关系型数据建模库还有 LightGBM、XGBoost 等。使用 LightGBM 库建立鸢尾花分类模型，并与 SVM 分类器的分类准确率进行比较。安装 LightGBM 库的命令如下：

```
pip install lightgbm
```

建模的范例代码如下：

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
from lightgbm import LGBMClassifier

# 假设有一个 DataFrame，其中包含特征('feature1', 'feature2', ..., 'featureN')和目标
# 变量('target')
data=pd.read_csv('your_data.csv')  # 请用自己的数据文件路径替换'your_data.csv'

# 划分训练集和测试集
X=data.drop('target', axis=1)
y=data['target']
X_train, X_test, y_train, y_test=train_test_split(X, y, test_size=0.2, random_
state=42)

# 初始化模型
model=LGBMClassifier()

# 训练模型
model.fit(X_train, y_train)

# 预测测试集结果
y_pred=model.predict(X_test)

# 计算预测准确率
accuracy=accuracy_score(y_test, y_pred)
print(f'Accuracy:{accuracy}')
```

习题解析